CREATE transaction simply specifies who the person is who is registering the object (this is usually the same as the initial owner of the asset).

An asset can represent any physical or digital object. It can be a car or a house, or it can be a digital object like a customer order or an air mile. An asset can have one or multiple owners, but it can also be its own owner.

An asset can be many different things, such as:

- A claim
- A token
- A versioned document
- A time series
- A state machine
- A permission (RBAC or role-based access control)

With TRANSFER transactions, complex instructions can be created that link existing assets (and their shares) to conditions of further transactions. In this way, assets and parts of the network can be easily moved between subscribers in the network. As is usual in a blockchain, there is no way to delete an asset or modify its properties.

Conditions for transactions

For a transaction to be validly processed in the network, several conditions must be met.

Once the transaction has been received, the nodes check it for the correct structure. For example, CREATE transactions must contain their asset data while TRANSFER transactions reference the ID of an asset created. Both types also differ in the way they have to deal with inputs and outputs. Of course, each transaction must be signed and hashed before it is transmitted to the network.

If the structure of a transaction is valid, the validity of the contained data is checked. In short, 'double spending' is prevented in this step. This prevents the transaction from repeatedly transferring the same asset or issuing assets that have already been issued in other transactions.

In addition, you can implement your own validation mechanisms that could check asset generation and transfers for correct functionality — for example, whether realistic dimensions have been assigned to a car part or whether the colour code of an automotive paint job exists.

Features of BigChainDB

The following are the features of BigChainDB.

- **Decentralised control:** BigchainDB has no single point of control and enjoys 100 per cent decentralised control with no single point of failure.
- **Query engine:** BigchainDB is backed by MongoDB to search for all contents in stored transactions, assets, metadata and blocks.
- **Tamper-resistant:** BigchainDB is 100 per cent fool-proof and tamper-resistant, i.e., once data is stored, it cannot be updated or deleted.
- **Byzantine fault tolerance:** Up to one third of the nodes in the network can experience arbitrary faults and the rest of the network will still come to a consensus on the next block.
- **Low latency:** It has a powerful database with low latency. Transactions happen very quickly.
- **Highly customised:** Helps the end users to design their own private network with custom assets, transactions, permissions and transparency.
- **Advanced access control:** Enables setting permissions at the transaction level to ensure the right set of duties and selective access.
- **Open source:** Fully open source, and anyone can build applications on top of the stack of BigchainDB.

Table 1

Point of difference	Hyperledger Fabric	BigchainDB
Operational interface	Difficult for newbies to understand and start off with.	Very easy to understand and start off with.
Types of nodes	It has different types of nodes (peers – which are also endorser and orderer nodes), which provide a flexible setup for organisations.	One type of node and a limited setup.
Types of transactions	Provides rich capabilities for doing all types of transactions. One kind of transaction can always be implemented by a custom processor function doing whatever is needed to query or modify the state of the ledger.	CREATE and TRANSFER transactions for every defined asset.
Types of APIs	More APIs and config models, and these are mostly implemented by the Composer framework.	No concept of APIs in BigchainDB.
Consensus	It uses Byzantine fault tolerance practically.	Uses Tendermint, which is Byzantine fault tolerant.